

Shortest Time to Completion First (STCF)

STCF runs the process with the least CPU work left. Unlike SJF, it can interrupt a running process when a shorter one becomes ready.

1. Model

The visualizer uses one CPU, whole ticks, and CPU-only processes. **Remaining service** is the number of CPU ticks a process still needs. It starts at the input service time and decreases by one for each tick the process runs. Switching processes takes no time.

A process can run only after it arrives. A ready process is waiting for the CPU; a future process has not arrived yet. Work already completed is preserved when a process is interrupted. Queue lists show insertion order. The short examples under the rules are separate scenarios.

2. Scheduling rules

Rule 1. Select the shortest remaining service

When the CPU is free, compare all ready processes and choose the one with the least remaining service. Use what remains now, not the original service time.

Explanation. A partially completed process may be shorter than a newly arrived one, even if it originally needed more CPU time. Selection searches the whole ready queue rather than taking its first entry.

Example. A originally needed ten ticks and has already received eight before being preempted. At the next dispatch, A has two ticks left and B has three. Select A. Its original ten-tick service is no longer the quantity being compared.

Rule 2. Preempt only for a strictly shorter process

If a ready process has less remaining service than the current process, interrupt the current process and select the shortest ready alternative. If their remaining times are equal, keep the current process running.

Explanation. First charge the tick that just ran, then compare the updated remaining times. Equality does not cause a switch. STCF has no quantum that forces equally short processes to alternate.

Example. A starts at time 0 needing five ticks. At time 2, it has three ticks left. Consider two separate arrivals for B:

<i>B's service at time 2</i>	<i>Comparison</i>	<i>Decision</i>
3 ticks	3 equals A's 3	A continues
2 ticks	2 is less than A's 3	B preempts A

Rule 3. Preserve work when a process is preempted

Append the interrupted process to the ready-queue tail without changing its remaining service. When selected again, it resumes from that amount.

Explanation. Preemption pauses a process; it does not restart its job. Returning to the queue tail also does not force it to wait for every entry ahead of it, because STCF still selects by remaining service.

Example. A and C arrive at time 0 needing six and eight ticks. At time 2, A has four left and C is waiting. B arrives needing one tick and preempts A. After B is dispatched, the queue is [C, A]. When B finishes at time 3, select A's four remaining ticks ahead of C's eight. A resumes with four, not six.

Rule 4. Break dispatch ties by arrival, then input order

When the CPU is free and several processes share the shortest remaining time, choose the earlier arrival. If arrivals also match, choose the one listed first in the input.

Explanation. These tie-breakers apply when choosing a process for a free CPU. They do not displace an equally short process already running; Rule 2 keeps that process on the CPU.

Example. At a dispatch boundary, B and C each have three ticks left. B arrived at time 1 and C at time 2, so select B. If two tied processes also arrived together, input order decides: Z listed before A is selected before A. Alphabetical order is irrelevant.

Rule 5. Complete at zero; idle only if nothing is ready

Remove a process when its remaining service reaches zero. Another process may start at that same boundary. If none is ready, wait for the next arrival. Stop after all processes finish.

Explanation. A completed process is removed before comparing candidates. If no candidate exists, the CPU stays idle until an arrival supplies one. There is no additional tick for completion or dispatch.

Example. A arrives at time 0 needing two ticks. B arrives at time 4 needing one, and C at time 5 needing one. A finishes at time 2; the CPU is idle until B arrives. B finishes at time 5, exactly when C arrives, so C runs immediately and completes at time 6.

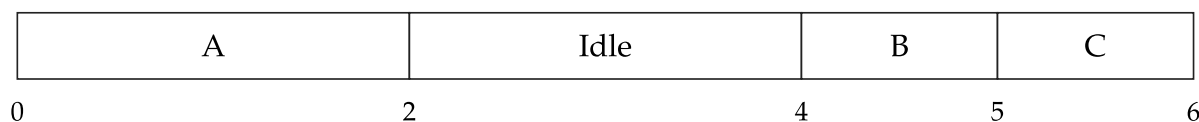


Figure 1. CPU schedule (time in ticks).

3. Events at a tick boundary

After accounting for the preceding tick:

1. Remove the running process if it has finished.
2. Admit all arrivals at the current time, in input order.
3. If a process is still running, check whether any ready process is strictly shorter. If so, append the interrupted process to the ready queue.
4. If the CPU is free, select the shortest remaining service, using Rule 4 for ties.

All simultaneous arrivals are considered before selecting a process. New arrivals enter the queue before an interrupted process is appended.

In Rule 3's example, C is already waiting when B arrives. Before dispatch, the queue becomes [C, B, A]: B is admitted first, then the preempted A is appended. B is selected from the middle because it has the least remaining service.

4. Worked example

Input order is A, B, C. All times are in ticks.

<i>Process</i>	<i>Arrival</i>	<i>Service</i>
A	0	5
B	1	2
C	1	1

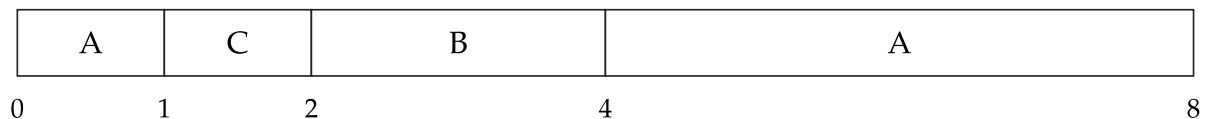


Figure 2. CPU schedule (time in ticks).

A starts at time 0. At time 1, it has four ticks left. Both B and C arrive, and C is shortest with one tick. C preempts A and finishes at time 2. B then runs for two ticks. A resumes at time 4 and uses its remaining four ticks, completing at time 8.

Notice that B does not get a turn merely because it appears before C in the input. Input order matters only after remaining time and arrival time are tied.